# Chapter 10 In-Class Exercise: Which Lookup Is Faster?

Prof. Jordan C. Hanson

## 1 Build a Large Student Roster

Suppose student IDs begin at

```
first = 2601001
N = 10000
```

Create a list of student names:

```
names = ["Student_"+str(i) for i in range(N)]
```

Now represent exactly the same information in two ways. First, create a **nested list**:

```
student_list =
[
    [first+i, names[i]] for i in range(N)
]
```

Then create a **dictionary**:

```
student_dict =
{
    first+i: names[i] for i in range(N)
}
```

Both structures contain the same ID–name pairs.

## 2 Lookup Using a Nested List

To find a student in the nested list, Python must examine records until it finds the correct ID.

```
def list_lookup(students,target_id):
    for student in students:
        if student[0] == target_id:
            return student[1]
    return None
```

Test the function:

```
target_id = first_id + N - 1
print(list_lookup(student_list,target_id))
```

## 3 Lookup Using a Dictionary

A dictionary uses the student ID directly as a key:

```
def dict_lookup(students,target_id):
    return students.get(target_id)
```

Test it:

```
print(dict_lookup(student_dict,target_id))
```

Which method is faster?

## 4 Measuring Execution Time

Python's `time` module contains a high-resolution timer called `perf_counter()`. Begin with `import time`. A single lookup happens so quickly that it may be difficult to measure accurately. Therefore, repeat the lookup many times.

```
repeats = 1000
```

Time the nested-list lookup:

```
start = time.perf_counter()
for i in range(repeats):
    list_lookup(student_list,target_id)
end = time.perf_counter()
list_time = end - start
```

Now time the dictionary:

```
start = time.perf_counter()

for i in range(repeats):
    dict_lookup(student_dict,target_id)
end = time.perf_counter()
dict_time = end - start
```

Display the results:

```
print("List:", list_time)
print("Dictionary:", dict_time)
```

## 5 Compare the Results

Compute how many times faster one method is:

```
ratio = list_time / dict_time
print("Ratio:", ratio)
```

Record your results:

$$t_{\text{list}} = _____$$

$$t_{\text{dict}} = _____$$

$$\frac{t_{\text{list}}}{t_{\text{dict}}} = _____$$

Repeat the experiment using

```
N = 100
N = 1000
N = 10000
```

For each value of $N$, search for the **last** student.